

L Y N X F A C T O R Y

Cómo tu web viaja de tu portátil al mundo entero

Recorrido completo — no asume conocimientos previos de DNS, CDN, webhooks o hashing.

Incluye diagramas, explicaciones en lenguaje claro, solución de problemas y glosario.

Lynx Factory

GitHub

Cloudflare

Arsys

editar → commit → push → build → publicar

Pipeline de despliegue web — edición española · v2.0
2026-05-17 · Borja Tarraso <borja.tarraso@member.fsf.org>

¿Qué es este documento?

Cuando modificas un archivo de una de tus webs (galeriaomaso.com, enriquetahueso.com, etc.) varios sistemas tienen que cooperar antes de que ese cambio aparezca en el navegador de un visitante en cualquier parte del mundo. Esta guía explica cada uno de esos pasos en lenguaje claro.

Está dirigida a dos públicos. (1) El operador actual que usa Lynx Factory a diario y quiere la imagen completa de lo que ocurre por debajo. (2) Una persona recién llegada que ha heredado el proyecto, que nunca ha usado Cloudflare ni GitHub, y que necesita levantar todo el stack desde una máquina limpia.

Cómo está estructurada esta guía

Orientación — qué es este documento y los cuatro sistemas involucrados.

Instalación desde cero — cuentas, tokens con privilegios mínimos, nameservers, instalación de Lynx.

Cómo funciona el pipeline — diagramas, narrativas, botón Test, watcher.

Operación diaria — añadir sitios, comandos habituales, recorrido por los archivos.

Análisis del fallo — por qué una versión anterior del botón Test fallaba en silencio y qué hace de distinto la nueva.

Procedimientos de emergencia, solución de problemas, glosario, fichas de referencia.

CONSEJO

Si un término te confunde, salta al glosario que está cerca del final. Esta guía no exige lectura lineal.

NOTA

Cada comando ha sido probado en Fedora Linux. Son comandos POSIX y funcionan en macOS y WSL con diferencias menores (nombres de los gestores de paquetes).

Los cuatro sistemas involucrados

1. Lynx Factory — el orquestador (vive en tu portátil)

Un panel de control basado en Flask que usas para escribir, almacenar y operar cada proyecto que tengas. Es el único sitio en el que 'pulsas un botón' para desplegar. Lynx habla con GitHub, Cloudflare y tus archivos locales.

2. GitHub — control de versiones (vive en github.com)

Almacena cada versión del código fuente de tu web. Hacer push de una versión nueva a GitHub es la señal canónica de que algo ha cambiado.

3. Cloudflare — build · caché · CDN (vive en 200+ ciudades)

Recibe el código nuevo (vía la CLI wrangler desde tu portátil, o vía la integración de auto-deploy de Cloudflare con GitHub), lo empaqueta como un Worker con assets estáticos, lo distribuye a los POPs de borde del mundo entero y lo sirve. Cloudflare actúa además como DNS autoritativo del dominio.

4. Arsys — tu registrador de dominio (vive en España)

Posee el derecho al nombre 'galeriaomaso.com' y similares. Arsys delega la resolución a los nameservers de Cloudflare con un único ajuste de configuración. Después de eso, Arsys queda en segundo plano hasta la renovación anual.

NOTA

Sin Lynx → no hay despliegue cómodo. Sin GitHub → no hay historial. Sin Cloudflare → no hay hosting, caché ni distribución global. Sin Arsys → ningún nombre de dominio resuelve a nada.

Acabo de heredar esto — ¿por dónde empiezo?

Bienvenido/a. Ahora eres responsable de una pequeña flota de webs y del tooling que las rodea. Antes de tocar nada, sigue esta orientación:

Paso 1 — Lee las dos primeras secciones

Necesitas un modelo mental de los cuatro sistemas antes de empezar a cambiar nada. Son 5 minutos.

Paso 2 — Consigue acceso a cada cuenta

bullet Cloudflare (dash.cloudflare.com) — hosting + CDN. Pide al operador anterior: email, contraseña (gestor de contraseñas) y códigos de respaldo 2FA.

bullet GitHub (github.com) — código fuente. Necesitas o bien acceso push a los repos, o bien el rol org-admin.

bullet Arsys (clientes.arsys.es) — registrador de dominios.

bullet Servidor / portátil donde corre Lynx Factory — SSH o acceso físico.

Paso 3 — Localiza los archivos de configuración existentes

bullet web/data/websites.json — lista de las webs que Lynx Factory conoce.

bullet .bashrc.d/lynx_factory_cf.sh — variables de entorno de Cloudflare (tokens scoped por proyecto, account ID, zone IDs).

bullet Los tokens de la API de Cloudflare en sí (en CF dashboard → My Profile → API Tokens).

Paso 4 — Verifica que nada esté roto ahora mismo

```
# ¿está Lynx Factory funcionando?
ss -ltnp | grep 9999

# ¿responde el watcher?
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | head -20

# ¿responden las webs en producción?
curl -sI https://www.galeriaomaso.com | head -1
curl -sI https://www.enriquetahueso.com | head -1
```

AVISO

No hagas push de ningún cambio a los repos de las webs hasta que puedas pulsar el botón Test en Lynx Factory y verlo parpadear verde. Eso confirma que todo el pipeline (portátil → wrangler/GitHub → Cloudflare → producción) está sano.

Qué necesitas instalado + cuentas que crear

Software en tu máquina

Python 3.11+ — ejecuta la app Flask de Lynx Factory y el watcher.

git — control de versiones.

Node.js + npx — los usa wrangler. El botón Test invoca `npx wrangler@latest deploy`.

Un navegador (Firefox/Chromium/Chrome/Brave/Edge/Vivaldi). El botón Test usa uno de ellos en modo incógnito.

Un editor a tu gusto (VS Code, Emacs, Vim).

Opcional pero recomendado: ripgrep, jq, httpie para inspeccionar JSON y HTTP.

```
# Fedora / RHEL
sudo dnf install -y python3 python3-pip git nodejs npm ripgrep jq

# Debian / Ubuntu / WSL
sudo apt install -y python3 python3-pip git nodejs npm ripgrep jq

# macOS (con Homebrew)
brew install python git node ripgrep jq
```

Cuentas que necesitas

Cloudflare (el plan gratuito basta para hosting estático) — dash.cloudflare.com.

GitHub — github.com. Activa 2FA.

Arsys (o el registrador donde tengas los dominios) — clientes.arsys.es.

Paquetes Python usados por Lynx Factory

Lynx Factory usa módulos de la librería estándar siempre que puede. Las pocas dependencias de terceros que necesita son:

```
pip install --user flask requests werkzeug

# Para generar estos documentos:
pip install --user matplotlib pillow reportlab
```

NOTA

Lynx Factory evita deliberadamente frameworks pesados. La app Flask, el thread watcher y el data store basado en JSON-en-disco mantienen todo inspeccionable con cat, grep y curl.

Cuenta Cloudflare + tokens API con privilegios mínimos

1. Crea una cuenta de Cloudflare

Abre `dash.cloudflare.com` → Sign up. Confirma tu email.

Activa 2FA (My Profile → Authentication).

2. Añade tu zona (dominio) a Cloudflare

Dashboard → Add a site → introduce `tudominio.com` → plan Free vale.

Cloudflare escanea los registros DNS existentes. Revisa, pulsa Continue.

Cloudflare te da dos nameservers (p.ej. `alice.ns.cloudflare.com` y `bob.ns.cloudflare.com`) — guárdalos para el paso de Arsys.

3. Encuentra tu Account ID y tu Zone ID

Cualquier URL del dashboard tiene la forma `dash.cloudflare.com/<ACCOUNT_ID>/<zone-name>/...` — ese es tu Account ID. El Zone ID se muestra en la página Overview de la zona (barra lateral derecha).

```
# ejemplo – sustituye con tus IDs reales
Account ID: 9f8b1c0e3d2f4e5a6b7c8d9e0f1a2b3c
Zone ID:    1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d
```

4. Crea un token scoped por proyecto (privilegio mínimo)

En vez de un único token todopoderoso, crea uno por web con exactamente los permisos que el despliegue de esa web necesita:

Nombre: `token-scoped-refresh-<proyecto>` (p.ej. `token-scoped-refresh-galeriaomaso`).

Permisos: Account → Workers Scripts → Edit + Zone → Cache Purge → Purge.

Account resources: Include → solo esta cuenta.

Zone resources: Include → Specific zone → la zona de este sitio.

Continue → Create Token → copia el valor una vez (no podrás verlo de nuevo).

5. Guarda los tokens donde Lynx los encuentre

```
# añade en un archivo que .bashrc sourcea en cada shell:
mkdir -p ~/.bashrc.d
cat > ~/.bashrc.d/lynx_factory_cf.sh <<'EOF'
export CF_ACCOUNT_ID="<tu-account-id>"
export CLOUDFLARE_ACCOUNT_ID="$CF_ACCOUNT_ID"

export CF_API_TOKEN_GALERIAOMASO="<token-scoped-1>"
export CF_API_TOKEN_ENRIQUETAHUESO="<token-scoped-2>"

export CF_ZONE_ID_GALERIAOMASO="<zone-id-1>"
export CF_ZONE_ID_ENRIQUETAHUESO="<zone-id-2>"
EOF
chmod 600 ~/.bashrc.d/lynx_factory_cf.sh

# cárgalo ahora en esta sesión:
source ~/.bashrc.d/lynx_factory_cf.sh
```

PELIGRO

El archivo de tokens contiene secretos. chmod 600. Nunca lo subas a git. Si un token se filtra, revócalo desde Cloudflare inmediatamente y crea uno nuevo scoped — nunca lo reemplaces por uno más poderoso.

CONSEJO

Mantén cualquier token 'poderoso' (multi-proyecto, alcance amplio) estrictamente bajo control humano. Lynx Factory debe correr siempre con tokens scoped por proyecto.

Conectar Cloudflare con GitHub (una vez, opcional)

Cloudflare puede construir y desplegar automáticamente cada vez que haces push a GitHub. Es cómodo pero por sí solo es poco fiable — la integración puede desconectarse en silencio. El camino basado en wrangler de Lynx Factory no depende de él, pero puedes mantenerlo activado como ruta paralela.

1. Crea el proyecto Pages/Workers

- Dashboard → Workers & Pages → Create application → Connect to Git.
- Pulsa Connect GitHub. Aparece un popup para instalar la Cloudflare GitHub App.
- Autoriza 'All repositories' o el repo específico.
- Vuelves al dashboard de Cloudflare.

2. Configura el build

- Production branch: main
- Framework preset: 'None' para HTML estático, o el de tu SPA.
- Build command: vacío para estático, p.ej. 'npm run build' para SPA.
- Output directory: '/' para estático, p.ej. 'dist'.
- Pulsa Save and Deploy.

3. Conecta el dominio personalizado

- Project → Custom domains → Set up a custom domain.
- Introduce www.tusitio.com. Repite para el ápice.

4. Verifica que el webhook de GitHub esté activo

- GitHub repo → Settings → Webhooks. Debe aparecer 'cloudflare-pages' con un check verde.
- Haz un commit minúsculo; en segundos debe aparecer un nuevo build en CF Dashboard → Deployments.

AVISO

Si los pushes dejan de disparar despliegues en silencio, este webhook es lo primero a sospechar. Reinstalar la Cloudflare GitHub App lo restaura. Mientras tanto el watcher + botón Test de Lynx Factory evitan el webhook llamando a wrangler directamente.

Configuración de nameservers en Arsys (una vez por dominio)

Arsys posee el registro del dominio pero la resolución DNS está delegada en Cloudflare. Se hace una vez por dominio.

1. Inicia sesión en Arsys

bullet Entra en clientes.arsys.es.

bullet Navega a Mis productos → Dominios → pulsa tudominio.com.

2. Reemplaza los nameservers

bullet Busca el panel 'Servidores DNS' / 'Nameservers'.

bullet Cambia de 'DNS Arsys' a 'DNS externos'.

bullet Pega los dos nameservers que te dio Cloudflare.

bullet Guarda. Confirma que la propagación puede tardar 24–48 horas.

3. Espera la propagación

```
# comprueba desde tu portátil – debería mostrar los NS de CF:
dig +short NS tudominio.com
# esperado tras la propagación:
# alice.ns.cloudflare.com.
# bob.ns.cloudflare.com.
```

4. Confirma que Cloudflare detecta el cambio

En dash.cloudflare.com → Overview, el banner de estado cambia de 'Pending Nameserver Update' a 'Active' (verde). Suelen ser pocas horas.

5. Mantén la auto-renovación en Arsys

Cloudflare gestiona el DNS pero Arsys sigue siendo el dueño del dominio. Activa la auto-renovación o pon un recordatorio anual. Un dominio caducado tira la web entera — ni Cloudflare ni Lynx pueden evitarlo.

PELIGRO

Nunca cambies los nameservers de vuelta a 'DNS Arsys' a menos que hayas movido el hosting fuera de Cloudflare. Hacerlo tira la web en minutos.

Lynx Factory — instalación + primer arranque

1. Clona el repositorio

```
git clone <repo-url> ~/claude/lynx_factory
cd ~/claude/lynx_factory
```

2. Instala dependencias

```
pip install --user flask requests werkzeug matplotlib pillow reportlab
```

3. Sourcea las variables de entorno

```
source ~/.bashrc.d/lynx_factory_cf.sh
echo $CF_API_TOKEN_GALERIAOMASO | cut -c1-8 # sanity check
```

4. Arranca Lynx Factory

```
# en primer plano (útil para el primer arranque; muestra logs):
./bin/lynx-factory --serve

# en segundo plano (daemonizado):
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &
disown

# abre la UI en tu navegador por defecto:
./bin/lynx-factory --browser
```

5. Verifica que responde

```
ss -ltnp | grep 9999
curl -s http://127.0.0.1:9999/ | head -5
```

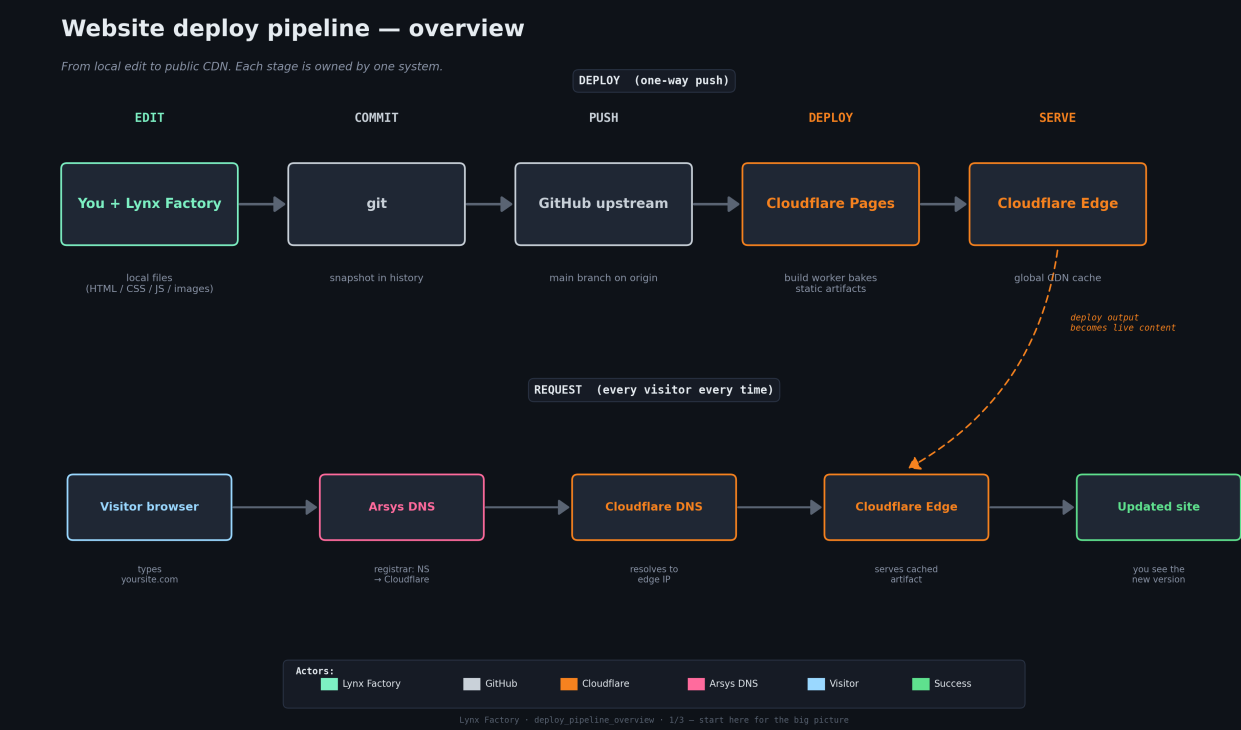
6. Abre el multiplexer

Visita <http://127.0.0.1:9999/multiplexer/web> — deberías ver una celda por web con botones RST · EXEC · Test arriba. Pulsa Test en una y observa el flujo de verificación.

CONSEJO

Si Lynx Factory se queja de variables de entorno faltantes al arrancar, olvidaste hacer source de `~/.bashrc.d/lynx_factory_cf.sh` en la misma shell desde la que lo lanzaste. Abre una terminal nueva y prueba otra vez — `.bashrc` auto-sourcea ese archivo.

Diagrama 1 — Visión general de los flujos deploy + request



Un cambio, de principio a fin (versión narrada)

Imagina que acabas de editar la imagen del banner de galeriaomaso.com en tu portátil. Esto es exactamente lo que sucede, paso a paso, hasta que un visitante en otro país ve ese nuevo banner.

Paso 1 — Guardas el archivo (tu portátil)

Sustituyes la imagen antigua por una nueva. En este punto nada ha llegado todavía a internet. El cambio existe solo en tu disco duro.

Paso 2 — Haces un commit en git (tu portátil)

git toma una instantánea: 'a fecha de ahora, este archivo contiene estos bytes'. La instantánea vive en una carpeta oculta llamada `.git`, todavía completamente local.

Paso 3 — git push (portátil → GitHub)

Subes la instantánea nueva a GitHub. GitHub tiene ahora la misma versión que tú.

Paso 4 — Lynx Factory se da cuenta (en menos de 60 s)

El watcher de Lynx hace polling de `origin/main` para cada sitio. Cuando ve que el SHA se ha movido, despacha un despliegue según el `deploy_method` del sitio (normalmente `wrangler`).

Paso 5 — wrangler despliega en Cloudflare

``npx wrangler@latest deploy`` corre desde el directorio `public/` del sitio, sube el bundle y Cloudflare publica la nueva versión del Worker.

Paso 6 — Cloudflare distribuye a los POPs de borde

El nuevo bundle se propaga a 200+ data centers en el mundo. El POP más cercano sirve a cada visitante desde RAM en milisegundos.

Paso 7 — El verify loop confirma que el borde está fresco

Lynx vuelve a descargar el fingerprint asset, lo hashea y lo compara con el local. Cuando los SHA-256 coinciden, el despliegue se declara 'fresco'.

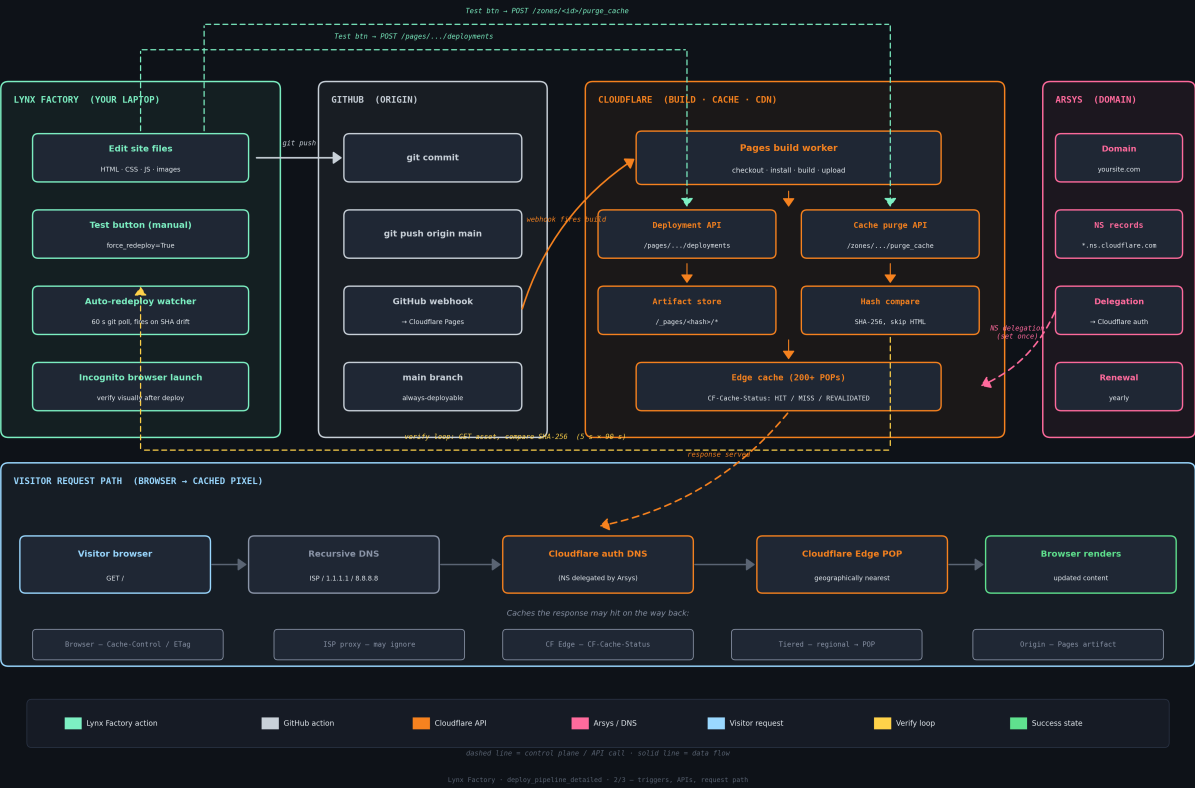
Paso 8 — Un visitante abre la página

Su navegador resuelve el dominio via `Arsys` → `Cloudflare DNS`, enruta al borde más cercano y el nuevo banner aparece.

Diagrama 2 — Vista detallada (triggers, APIs, capas de caché, saltos DNS)

Website deploy pipeline — detailed view

Triggers, APIs, cache layers, DNS hops. Two ways to deploy: GitHub push (auto) or Lynx Test button (manual).



Qué experimenta realmente el visitante

DNS — convertir un nombre en un número

Las máquinas no pueden hablar directamente con 'galeriaomaso.com' — necesitan una dirección IP. El Domain Name System traduce uno a otro. Cuando un navegador pregunta '¿dónde está galeriaomaso.com?' la respuesta salta:

- El navegador pregunta al resolver recursivo (ISP, 1.1.1.1, 8.8.8.8).
- El resolver pregunta a los TLD servers ('.com').
- Los TLD apuntan a Arsys (porque Arsys es el registrador).
- Arsys dice 'este dominio delega en *.ns.cloudflare.com'.
- El resolver pregunta al DNS de Cloudflare por la IP.
- El DNS de Cloudflare responde con la IP del POP de borde más cercano.

NOTA

Tras la primera consulta la respuesta queda cacheada en el resolver y en el navegador durante un tiempo. Por eso los cambios DNS pueden tardar en extenderse.

CDN — muchos servidores de borde, una sola web

Una Content Delivery Network es una flota global de pequeños data centers. Cada uno guarda una copia de tus archivos estáticos. El visitante en Tokio pega al borde de Tokio; el de Helsinki al de Helsinki — pero ven la misma web.

Capas de caché — dónde se esconde el contenido obsoleto

- Caché del navegador: el navegador del visitante guarda copias según las cabeceras Cache-Control.
- Proxy de ISP / corporativo: algunas redes cachean respuestas entre tú y Cloudflare.
- Caché de borde de Cloudflare: cada POP guarda respuestas recientes en RAM.
- Caché en tiers de Cloudflare: una caché regional entre los POPs y el origen.
- Origen: el bundle del Worker / store de assets estáticos (fuente de verdad).

AVISO

'Cambié el archivo pero mi móvil sigue viendo el antiguo' es casi siempre un problema de caché en alguna de esas cinco capas. El botón Test purga el caché de borde de Cloudflare pero no puede purgar la caché local de tu móvil — usa una pestaña incógnito.

El botón Test — qué hace exactamente

En cada celda de web del multiplexer hay un pequeño botón Test mint-teal. Pulsarlo ejecuta un health check de extremo a extremo que toca cada paso del pipeline.

Paso a paso

- Lee `websites.json` para encontrar `live_url`, `source_dir`, `public_subdir`, `fingerprint_asset`, `deploy_method` y credenciales de CF.
- Opcionalmente purga la caché de Cloudflare para esa zona (para que el siguiente fetch venga del origen).
- Descarga el fingerprint asset con un parámetro de cache-bust.
- Calcula el SHA-256 sobre la copia live y la copia local; compara.
- Si los hashes difieren → despacha un redeploy via `_dispatch_deploy()` (wrangler o pages_api).
- Verify loop: cada 5 s durante hasta 90 s, refetch + rehash hasta coincidir.
- Sondea un asset nuevo conocido (saltando archivos de config: `_headers`, `_redirects`, `_worker.js`, `wrangler.*`).
- Chequea git: ¿está el clon local sucio? ¿adelante/atrás del origen?
- Abre la live URL en una pestaña incógnito del navegador.

Modificadores de click

- Click normal → `force_redeploy=True` (limitado por staleness — no dispara si todo coincide).
- Shift + click → `force_unconditional=True` (siempre dispara un redeploy, ignora el hash).
- Alt + click → `open_browser=False` (no abre incógnito; útil para checks headless).

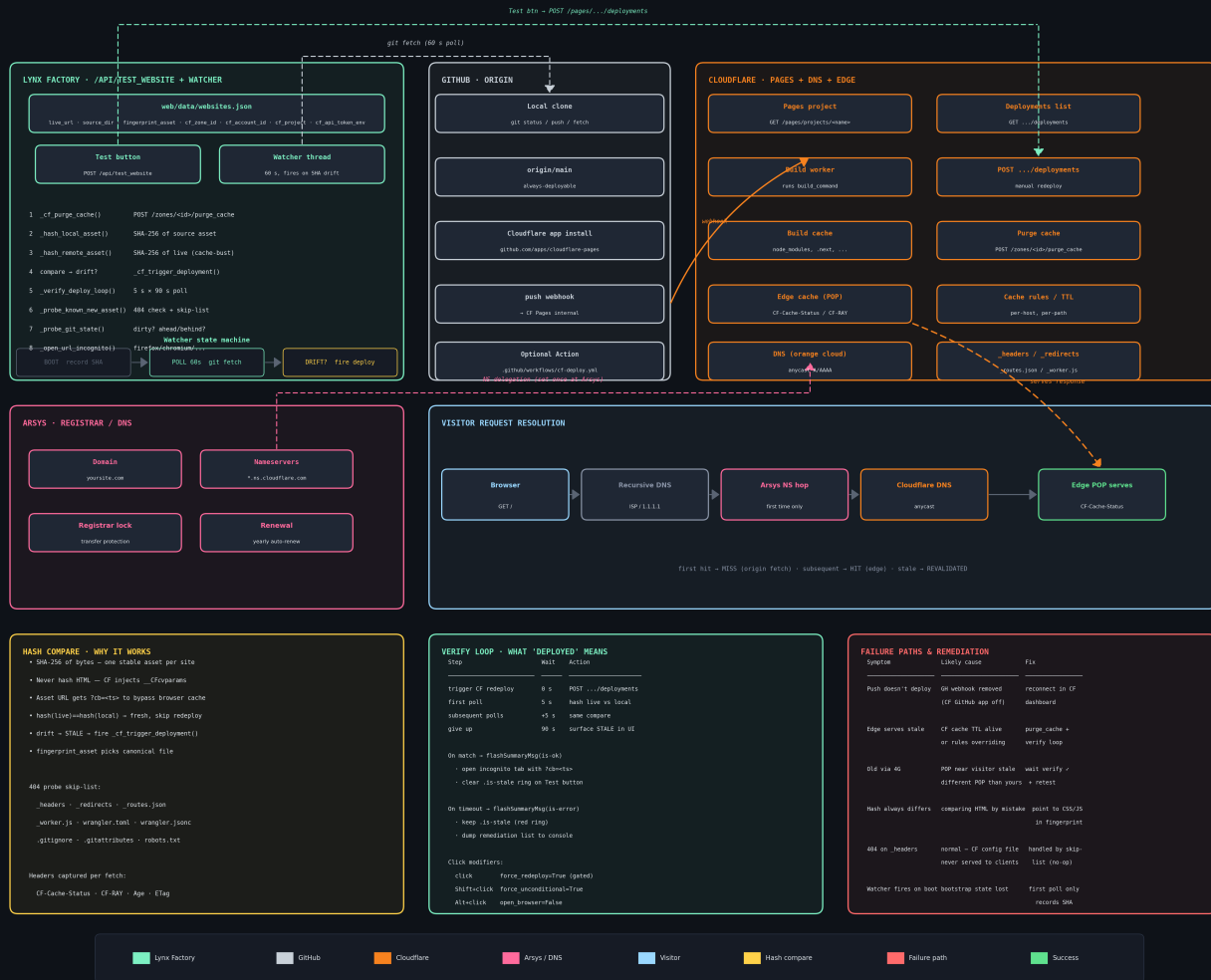
CONSEJO

Si el botón se queda rojo, el sitio live está obsoleto. Hover para tooltip; la consola del navegador tiene la lista completa de remediación.

Diagrama 3 — Internals completos (cada API, función, skip-list, modo de fallo)

Website deploy pipeline — full internals

Every function, API endpoint, state transition and skip-list used by the Test button and the auto-redeploy watcher.



El watcher de auto-redeploy

Lynx Factory tiene un thread en segundo plano que hace polling cada 60 segundos. Compara el último commit de origin/main contra el último commit que vio. Si hay drift, despacha un despliegue automáticamente (mismo dispatcher que el botón Test).

Por qué esto importa

El webhook de auto-deploy de Cloudflare con GitHub puede desconectarse en silencio (app desinstalada, token revocado, glitch de red). Cuando ocurre, los pushes parecen ir bien por tu lado pero no producen nunca un nuevo deploy. El watcher es la red de seguridad: si hiciste push, Cloudflare desplegará — de un modo u otro.

Máquina de estados

bullet BOOT → registra el SHA actual, nunca dispara (evita despliegues 'fantasma' en cada reinicio).

bullet POLL → cada 60 s: git fetch + git rev-parse origin/<rama>.

bullet DRIFT? → si el SHA remoto cambió, llama a _dispatch_deploy(meta, ...).

bullet Idle → si no hay drift, vuelve a POLL.

Dónde ver su estado

```
$ curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq
{
  "watcher": "running",
  "interval_s": 60,
  "sites": {
    "galeriaomaso": {
      "last_seen_sha": "ad376ff...",
      "last_trigger_method": "wrangler",
      "last_fired_at": "2026-05-17T13:42:11Z"
    }
  }
}
```

NOTA

El watcher requiere variables CF_API_TOKEN_<PROJECT> por proyecto y el campo cloudflare_account_id en websites.json. Sin ellas el watcher sigue haciendo polling pero no puede disparar un deploy.

Añadir una nueva web (receta completa)

Receta canónica para coger un dominio nuevo y servirlo a través del mismo pipeline Lynx + GitHub + Cloudflare + Arsys.

1. Registra el dominio en Arsys

Cómpralo en clientes.arsys.es. Espera el email de confirmación.

2. Añade la zona a Cloudflare

Dashboard → Add a site. Cloudflare te da dos nameservers.

3. Actualiza los nameservers en Arsys

Cambia a esos dos NS de CF. Espera la propagación.

4. Crea el repo en GitHub

```
# en github.com: crea repo vacío 'misitio'

cd ~/proyectos
git clone git@github.com:tu/misitio.git
cd misitio
echo '<h1>Hola mundo</h1>' > index.html
git add . && git commit -m 'primer commit'
git push -u origin main
```

5. Crea el proyecto Workers/Pages en Cloudflare

Para sitios estáticos con wrangler: añade un `wrangler.jsonc` en tu directorio `public/`. Primer deploy:

```
cd ~/proyectos/misitio/public
npx wrangler@latest deploy
```

Después configura el dominio personalizado `misitio.com + www` en el dashboard de CF.

6. Crea un token CF scoped para este proyecto

Nómbralo `token-scoped-refresh-misitio`. Permisos: Workers Scripts: Edit + Cache Purge: Purge, limitado a esta cuenta y zona.

```
echo 'export CF_API_TOKEN_MISITIO="<token-scoped>"' \
>> ~/.bashrc.d/lynx_factory_cf.sh
source ~/.bashrc.d/lynx_factory_cf.sh
```

7. Registra el sitio con Lynx Factory

```
// edita web/data/websites.json – añade una entrada:
{
  "live_url":      "https://www.misitio.com",
  "source_dir":    "/home/tu/proyectos/misitio",
  "public_subdir": "public",
  "fingerprint_asset": "style.css",
  "cloudflare_zone_id": "<zone-id>",
  "cloudflare_account_id": "<account-id>",
  "cloudflare_project": "misitio",
  "cloudflare_api_token_env": "CF_API_TOKEN_MISITIO",
  "deploy_method": "wrangler",
  "auto_redeploy_on_push": true,
  "auto_redeploy_branch": "main"
}
```

8. Reinicia Lynx Factory y verifica

```
pkill -f 'web/app.py'
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &

# abre el multiplexer, busca la nueva celda, pulsa Test.
```

CONSEJO

Elige un archivo CSS/JS/PNG como `fingerprint_asset`, no HTML. Cloudflare modifica el HTML en vuelo (script de bot-management), lo que rompe la comparación SHA-256.

Flujo de trabajo diario

Empujar un cambio a una web en producción

```
cd ~/proyectos/misitio
vim index.html
git add -p && git commit -m 'retoque del titular'
git push                                # el watcher despacha deploy en <60s

# 0 pulsa Test en el multiplexer para desplegar inmediatamente.
```

Forzar un redeploy aunque Lynx crea que nada cambió

En el multiplexer, Shift+click en el botón Test. Esto activa `force_unconditional=True` así que se dispara un deploy independientemente del hash.

Ver qué está haciendo el watcher

```
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq
```

Reiniciar Lynx Factory limpiamente

```
pkill -f 'web/app.py'
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &
disown
```

Hacer tail de los logs de Lynx Factory

```
tail -f /tmp/lynx-factory.log
```

Añadir una nueva variable de entorno (token CF, etc.)

```
echo 'export CF_API_TOKEN_NUEVOSITIO="..."' \
  >> ~/.bashrc.d/lynx_factory_cf.sh
source ~/.bashrc.d/lynx_factory_cf.sh
# reinicia Lynx Factory para que Flask herede la nueva variable
```

Hacer rollback a un deploy anterior

CF Dashboard → Workers & Pages → proyecto → Deployments → escoge un deploy anterior exitoso → Rollback. El caché de borde vuelve a intercambiarse en segundos.

Dónde vive todo — recorrido por archivos/código

Estructura de alto nivel

```
lynx_factory/
├── bin/
│   └── lynx-factory          # launcher
├── web/
│   ├── __main__.py          # CLI: --serve / --browser
│   ├── app.py               # Flask app + endpoints + watcher
│   ├── auth.py              # gate @before_request
│   └── data/
│       ├── websites.json    # metadata web + config CF
│       ├── logs.json        # log rotativo de actividad
│       └── ...
│   ├── static/
│       ├── multiplexer.js   # UI multiplexer + botón Test
│       ├── quota_panel.js   # barra semanal + panel de uso
│       └── style.css         # todo el estilo visual
│   └── templates/
│       ├── _base.html       # chrome de página
│       └── multiplexer.html  # celdas con RST/EXEC/Test
├── docs/website_deploy/
│   ├── generate_infographics.py # generador de diagramas PNG
│   ├── pdf_guide.py           # generador PDF ReportLab
│   └── deploy_pipeline_*.png, pdf # salidas generadas
└── prompts_in/ tracking/ steganography/
```

El botón Test — dónde vive el código

- Endpoint del servidor: web/app.py — /api/test_website.
- Dispatchers: _dispatch_deploy → _wrangler_deploy | _cf_trigger_deployment.
- Helpers de hash: _hash_remote_asset, _hash_local_asset, _verify_deploy_loop.
- Handler cliente: web/static/multiplexer.js — checkFreshedWebsite.
- HTML del botón: web/templates/multiplexer.html — mux-cell-btn-test.

El thread watcher

- AUTO_REDEPLOY_THREAD, _AUTO_REDEPLOY_STATE, _auto_redeploy_tick — todos en web/app.py.
- Lanzado por start_background_threads() al arrancar la app.
- Introspectable via GET /api/auto_redeploy_state.

Por qué el botón Test fallaba en silencio (y qué cambió)

Síntomas

Borja editó el banner de galeriaomaso.com, commiteó y empujó a GitHub. El dashboard de Cloudflare mostraba el push de GitHub pero ningún deploy nuevo. El botón Test en Lynx Factory reportaba éxito — pero la web live seguía sirviendo el banner viejo.

Causa raíz #1 — se asumió que el auto-deploy GitHub→CF estaba conectado

Se asumía que el proyecto estaba enganchado a la integración de Cloudflare con GitHub ('push a main → CF construye → CF publica'). No lo estaba. Todo deploy previo se había hecho manualmente con la CLI wrangler. No había GitHub App instalada para este proyecto — los pushes eran invisibles para Cloudflare.

Causa raíz #2 — endpoint API incorrecto para un proyecto Workers

El helper de redeploy de Lynx Factory llamaba al endpoint REST de Cloudflare Pages (POST `/accounts/<id>/pages/projects/<name>/deployments`). Pero [galeriaomaso](https://galeriaomaso.com) es un proyecto Workers (con el bundler de static-assets), no un proyecto Pages. Ese endpoint devolvía HTTP 404 en cada intento — silenciosamente, porque la respuesta se logueaba pero el botón Test solo reportaba el flag OK de alto nivel.

Causa raíz #3 — sin verificación de hash del fingerprint

El código Test previo declaraba éxito cuando la llamada HTTP a CF devolvía 2xx, independientemente de si algo había llegado realmente al borde. No había verificación de que los bytes en el sitio live hubieran cambiado.

Cuál era el comportamiento original de Cloudflare

Cloudflare hacía exactamente lo que le habían dicho: servir el bundle Worker desplegado más recientemente desde la caché de borde, indefinidamente. Como no había corrido un `wrangler deploy` nuevo desde el último deploy manual exitoso, el borde correctamente servía ese bundle viejo. El push de GitHub se ignoraba — no había integración instalada para reaccionar a él.

La solución

- bullet Añadir `_wrangler_deploy()` en `app.py`: corre ``npx wrangler@latest deploy`` dentro de `<source_dir>/<public_subdir>`, con `CF_API_TOKEN` + `CF_ACCOUNT_ID` inyectados via `env`.
- bullet Añadir `_dispatch_deploy()`: lee `deploy_method` desde `websites.json` (`wrangler` | `pages_api` | `github_auto`) y enruta al helper correcto.
- bullet Actualizados `/api/test_website` y `_auto_redeploy_tick` para llamar a `_dispatch_deploy` en lugar de la ruta solo-Pages.
- bullet Añadir un verify loop de fingerprint (5 s × 90 s) que re-hashea el asset live tras el deploy hasta que coincide con el local.
- bullet Migración de LF a tokens scoped por proyecto con privilegio mínimo (`token-scoped-refresh-<project>`).

CONSEJO

Tras cada deploy el botón Test reporta `cf_redeploy.method` (`wrangler` | `pages_api` | `github_auto`) y `verify.elapsed_s`. Si `method` es `wrangler` y `verify.ok` es `true`, los nuevos bytes están en el borde.

Procedimientos de emergencia

El sitio en producción está caído

- bullet Abre Cloudflare → Workers & Pages → proyecto → Deployments. ¿Está el último en rojo?
- bullet Si sí: pulsa Rollback en el anterior verde.
- bullet Si no: chequea CF → Overview. ¿Estado de la zona 'Active'?
- bullet Si ambos parecen bien: chequea `dns.google` o `cachecheck.cloudflare.com`.

Hiciste un commit malo y ya está pusheado

```
git revert HEAD --no-edit && git push
# el watcher redespliega via wrangler en <60s
```

Token API de Cloudflare filtrado

- bullet Cloudflare → My Profile → API Tokens → clicas el token → Roll.
- bullet Sustituye el valor en `~/bashrc.d/lynx_factory_cf.sh` por el nuevo.
- bullet Source el archivo y reinicia Lynx Factory.
- bullet Mantén el mismo scope (por proyecto, privilegio mínimo). No 'subas de nivel' a un token más amplio.

Cuenta de Cloudflare bloqueada

- bullet Recupera via códigos de respaldo 2FA.
- bullet Si los perdiste: `cloudflare.com/recovery` — requiere verificación de identidad, lleva días.
- bullet Mientras tanto los sitios siguen sirviendo — solo el dashboard está bloqueado, no el borde.

Dominio de Arsys a punto de expirar

- bullet `clientes.arsys.es` → Dominios → Renovar.
- bullet Un dominio caducado tira el sitio en horas. Activa auto-renovación + recordatorio anual.

Lynx Factory no arranca

```
tail -50 /tmp/lynx-factory.log
# causas comunes:
# - puerto 9999 ya ocupado (mata el python web/app.py viejo)
# - websites.json malformado (python -m json.tool web/data/websites.json)
# - dependencia pip faltante (rerun el pip install de arriba)
```

PELIGRO

Antes de cualquier comando destructivo, haz un snapshot: `cp -a web/data /tmp/lf-backup-$(date +%s)`
Lynx Factory guarda backups automáticos bajo `web/data/_backups/` pero uno extra nunca hace daño.

Cuando algo va mal — solución de problemas

'Hice push pero nada cambia en producción'

- bullet Pulsa Test en Lynx Factory. Si se pone rojo, el hash live difiere del local — el redeploy no llegó al borde.
- bullet Abre `dash.cloudflare.com` → Workers & Pages → proyecto → Deployments. ¿Hay un deploy reciente?
¿Tuvo éxito?
- bullet Si no hay ninguno, la integración auto-deploy con GitHub falta o está rota — pero la ruta wrangler de Lynx debería funcionar. Prueba Shift+click en Test para forzar el redeploy incondicional.
- bullet Chequea `/tmp/lynx-factory.log` para stderr de wrangler.

'Mi móvil muestra el sitio antiguo pero mi portátil muestra el nuevo'

- bullet El móvil tiene una copia cacheada. Ábrelo en una pestaña incógnito.
- bullet El móvil puede estar pegando a un POP de borde distinto. Espera un minuto, retry.
- bullet El Wi-Fi de casa puede tener caché DNS devolviendo respuesta vieja. Reinicia router o usa datos móviles.

'Funcionaba ayer, hoy no, no cambié nada'

- bullet Abre Cloudflare → Deployments. ¿Hubo un build reciente en rojo?
- bullet Si sí, el log del build suele mostrar qué cambió (dependencia, fuente, env).
- bullet Arregla y empuja otra vez, o rollback al anterior exitoso.

'El botón Test siempre está rojo aun esperando'

- bullet Comprueba que `CF_API_TOKEN_<PROJECT>` está exportada y visible al proceso Flask (reinicia LF tras setearla).
- bullet Comprueba que `cloudflare_account_id` está en `web/data/websites.json`.
- bullet Confirma que `fingerprint_asset` apunta a un archivo real bajo `source_dir` (y NO es HTML).
- bullet Verifica que el token scoped tiene Workers Scripts: Edit en esta cuenta — llama a `/accounts/<id>/tokens/verify` con él.

PELIGRO

Nunca elijas una página HTML como fingerprint asset. Cloudflare inyecta un pequeño snippet JS (`__CF$cv$params`) en respuestas HTML al vuelo, por lo que el HTML live nunca coincide con el HTML local por SHA-256.

Arquitectura en profundidad

Modelo de procesos

Lynx Factory es un único proceso Python. Flask corre en el hilo principal; varios threads daemon corren en background — el relevante aquí es el watcher de auto-redeploy. No hay worker separado, ni cola, ni scheduler. Todo está in-process.

Almacén de datos

Configuración y estado viven como archivos JSON bajo web/data/. Las lecturas son cacheadas por mtime para que las rutas calientes no re-parseen el archivo en cada request. Las escrituras son atómicas (escribir a .tmp, fsync, rename). Backups bajo web/data/_backups/ con retención rotativa.

Modelo de auth

web/auth.py instala un hook Flask @before_request que protege todos los endpoints excepto si la request lleva una cookie de sesión válida. El token vive en ~/.lynx/web-token; visitas /login?token=<paste> una vez por navegador.

Modelo de interacción con Cloudflare

- Operaciones de lectura (GET project, GET deployments) usan requests.get con timeout 10 s.
- Operaciones de escritura (POST purge_cache, POST deployments) usan requests.post con timeout 30 s.
- Deploys de wrangler corren via subprocess: npx --yes wrangler@latest deploy, timeout 300 s.
- Los tokens se leen del env en cada llamada (no cacheados) así que rotar un token requiere solo un restart de LF.

Por qué el verify loop hace polling y no webhooks

Cloudflare ofrece webhooks para estado de deploy pero requieren una URL públicamente alcanzable — tu portátil no lo es. Polling cada 5 s durante 90 s es más simple, más fácil de debuguear y suficiente — un deploy exitoso suele reflejarse en el borde en 10–30 s.

Por qué hashear en lugar de confiar en el status de Cloudflare

Un build 'exitoso' en Cloudflare no siempre significa que el borde sirva el nuevo contenido inmediatamente. Reglas de caché, caché en tiers, propagación regional pueden ir con retraso. Comparar los bytes reales que recibes contra los bytes en disco es la única verdad.

Glosario

- **Caché:** copia temporal de una respuesta mantenida cerca del solicitante para acelerar la próxima petición.
- **CDN (Content Delivery Network):** flota global de servidores de borde, cada uno con copias cacheadas de tus archivos estáticos.
- **CF-Cache-Status:** cabecera de respuesta de Cloudflare. HIT = servido desde caché de borde, MISS = traído del origen, REVALIDATED = el origen confirmó que la caché sigue fresca.
- **Commit:** instantánea guardada en git.
- **DNS (Domain Name System):** el sistema que convierte tudominio.com en una IP.
- **Edge POP:** un data center concreto de Cloudflare, uno de 200+ por el mundo.
- **Fingerprint asset:** un archivo estable (CSS/JS/imagen — nunca HTML) usado por el botón Test para detectar drift.
- **Hash (SHA-256):** fingerprint corto de longitud fija de bytes arbitrarios; dos entradas producen el mismo hash solo si son byte-idénticas.
- **Hook (webhook):** llamada HTTP que un sistema hace a otro cuando ocurre algo interesante.
- **Pestaña incógnito:** ventana de navegador que evita tu caché y cookies locales.
- **Token de privilegio mínimo:** token API con exactamente los permisos que la tarea necesita y ni uno más.
- **Rama main:** la rama por defecto 'always-deployable' en GitHub.
- **Nameserver:** servidor DNS autoritativo para un dominio. Arsys delega en los nameservers de Cloudflare.
- **Origen:** el almacén fuente-de-verdad de donde el CDN trae cuando no tiene copia cacheada.
- **Pages (Cloudflare Pages):** producto de hosting estático de Cloudflare. Construye al hacer push, sirve en el borde.
- **Push:** subir commits locales de git a un remoto (GitHub).
- **Purge:** borrar una copia cacheada para que la próxima request traiga una fresca.
- **Recursive resolver:** el servidor DNS al que tu máquina pregunta (1.1.1.1, 8.8.8.8).
- **Registrador:** la empresa que posee el derecho a un nombre de dominio (Arsys aquí).
- **Token scoped:** token cuyos Account / Zone Resources están restringidos a una sola cuenta y zona.
- **TTL (Time To Live):** cuánto tiempo se permite que una respuesta cacheada permanezca antes de refrescarse.
- **Verify loop:** el polling 5 s × 90 s del botón Test que confirma que un deploy ha llegado realmente al borde.
- **Watcher:** thread en background de Lynx Factory que auto-dispara un deploy cuando origin/main avanza.
- **Workers (Cloudflare Workers):** plataforma serverless de Cloudflare. Lynx envía Workers de static-assets via wrangler.
- **wrangler:** CLI de Cloudflare para gestionar Workers. Lynx la invoca via `npx wrangler@latest deploy`.

Referencia rápida — archivos, env vars, APIs

websites.json (campos por proyecto)

```
Obligatorios:
  live_url          https://www.galeriaomaso.com

Recomendados:
  source_dir        ruta al repo local
  public_subdir     subdir servido en / (default: public)
  fingerprint_asset ruta relativa (CSS/JS/imagen)
  cloudflare_zone_id de la URL del dashboard de CF
  cloudflare_account_id de la URL del dashboard de CF
  cloudflare_project nombre del proyecto CF
  cloudflare_api_token_env p.ej. CF_API_TOKEN_GALERIAOMASO
  deploy_method     wrangler | pages_api | github_auto
  auto_redeploy_on_push true / false
  auto_redeploy_branch main
```

Variables de entorno (~/.bashrc.d/lynx_factory_cf.sh)

```
CF_ACCOUNT_ID          un account ID para todos los sitios
CLOUDFLARE_ACCOUNT_ID  alias de CF_ACCOUNT_ID (wrangler lo lee)
CF_API_TOKEN_<PROJECT> token scoped por proyecto
CF_ZONE_ID_<PROJECT>   zone ID por proyecto (para cache purge)
```

Endpoints de la API de Cloudflare usados

```
POST /zones/<zone_id>/purge_cache
  body: {"purge_everything": true}

POST /accounts/<acc_id>/pages/projects/<name>/deployments
  usado por deploy_method=pages_api (solo proyectos Pages)

subprocess wrangler (deploy_method=wrangler):
  npx --yes wrangler@latest deploy
  cwd: <source_dir>/<public_subdir>
  env: CLOUDFLARE_API_TOKEN + CLOUDFLARE_ACCOUNT_ID
```

Endpoints de Lynx Factory

```
POST /api/test_website
  body: {
    "name": "galeriaomaso",
    "force_redeploy": true,      // default del botón Test
    "force_unconditional": false, // Shift+click pone true
    "open_browser": true       // Alt+click pone false
  }

GET /api/auto_redeploy_state # introspección del watcher
```

Checklist de configuración (resumen de una página)

En tu portátil

- Crea un token CF scoped por proyecto: token-scoped-refresh-<project>.
- Permisos: Workers Scripts: Edit + Cache Purge: Purge, limitado a esta cuenta y zona.
- Exporta cada uno como CF_API_TOKEN_<PROJECT> en ~/.bashrc.d/lynx_factory_cf.sh para que persista entre reboots.
- Rellena cloudflare_zone_id, cloudflare_account_id, cloudflare_project, cloudflare_api_token_env, deploy_method por sitio en web/data/websites.json.
- Reinicia Lynx Factory para que recoja los nuevos env vars y el JSON.

Verifica que todo funciona

```
# ¿token visible en la shell actual?
echo $CF_API_TOKEN_GALERIAOMASO | cut -c1-8

# ¿watcher de Lynx sano?
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq

# Pulsa Test en el multiplexer.
# Flash mint-teal = OK · rojo = obsoleto · consola tiene el reporte completo.
```

En Cloudflare

- (Opcional) instala la Cloudflare GitHub App para auto-deploy basado en push. La ruta wrangler funciona sin ella.
- Confirma que aparece un deploy reciente en Workers & Pages → proyecto → Deployments.

En Arsys

- Confirma que los nameservers del dominio apuntan a *.ns.cloudflare.com.
- Confirma que la renovación del dominio está en auto-renovar (o tiene un recordatorio manual).

CONSEJO

Una vez todo está configurado, casi nunca deberías necesitar abrir el dashboard de Cloudflare. El botón Test + watcher de Lynx cubre el bucle deploy + verify diario.